MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers Informatics and Microelectronics

Department of Software Engineering and Automation

Report on laboratory work 4

The xv6 shell from the outside

Course Operating Systems

Done by: st. gr. FAF-242, Tacu Igor
Checked by: Reițman Patricia

Chișinău, 2026

Purpose and execution record

The experiments were repeated on the Ubuntu x86_64 server on 9 November 2026. The xv6 tests use an isolated source copy and fresh generated disk. Figures are browser screenshots of a read-only execution-record console displaying saved SSH output; the exact text logs and source are included in the repository. Predictions were written to predictions.json before each corresponding command was run. The submission was prepared with coding assistance, and MIT xv6 is identified as upstream in the source provenance file.

Part 1 Sleep and the current interface

The header declares pause rather than sleep. The submitted utility calls pause with validated decimal ticks; sleep 20 returns normally, missing input produces usage, and negative input produces a diagnostic. The Makefile contains the _sleep entry. Forked processes inherit the binary and descriptor context, while pause is the kernel operation that performs the delay.

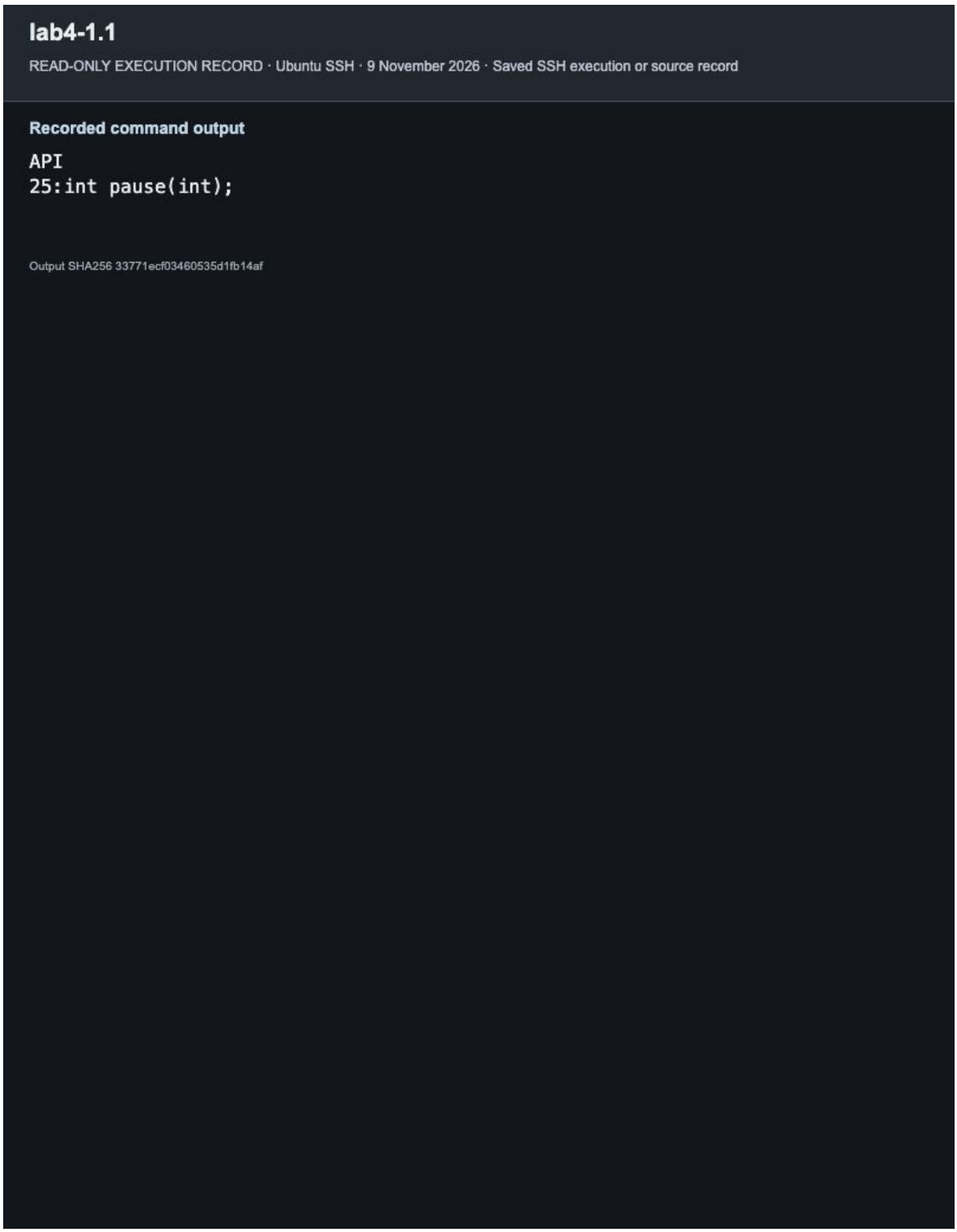


Figure 1.1 – The pause declaration

lab4-1.2

READ-ONLY EXECUTION RECORD · Ubuntu SSH · 9 November 2026 · Saved SSH execution or source record

Recorded command output

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char **argv)
{
  if(argc != 2 || argv[1][0] == 0){
    fprintf(2, "usage: sleep nonnegative_ticks\n");
    exit(1);
  }
  int ticks = 0;
  for(char *p = argv[1]; *p; p++){
    if(*p < '0' || *p > '9' || ticks > (2147483647 - (*p - '0')) / 10){
      fprintf(2, "sleep: invalid tick count\n");
      exit(1);
    }
    ticks = ticks * 10 + (*p - '0');
  }
  exit(pause(ticks) < 0);
}

UPROGS
134:    $U/_pingpong\
135:    $U/_run\
136:    $U/_find\
137:    $U/_mystat\
138:    $U/_sleep\
139:    $U/_args\
```

Output SHA256 5be3aee02e0f3dbf91dc645c

Figure 1.2 – Sleep source and registration

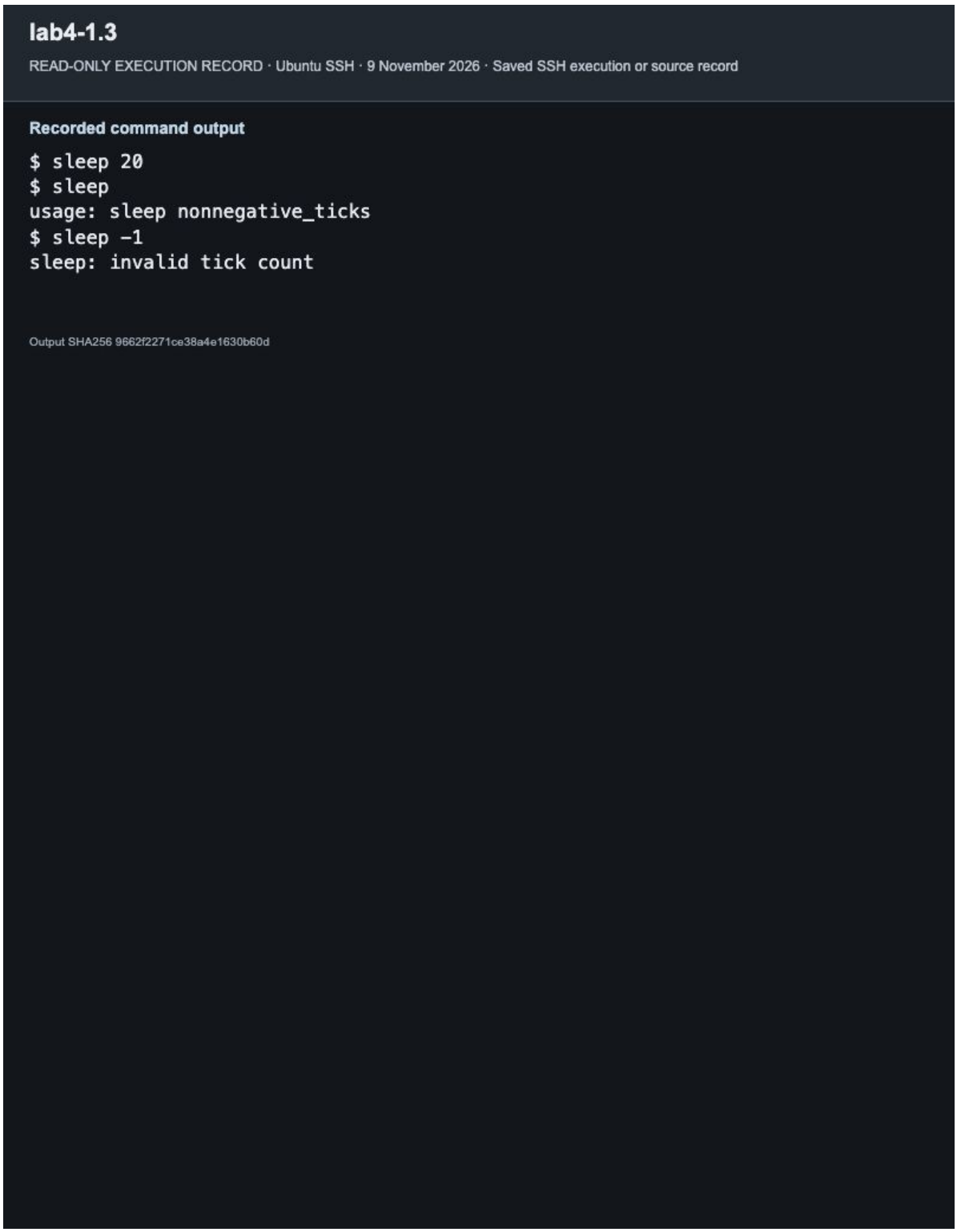


Figure 1.3 – Sleep with valid and invalid arguments

Part 2 Comparing every command in the two shells

The Bash test starts in a fresh directory containing README. Both shells provide execution, descriptor redirection, pipelines, command lists, grouping and background commands. Bash additionally expands wildcards and variables, removes syntactic quotes, and offers readline history and completion. The table records the actual small-shell behavior rather than assuming it implements Bash syntax.

Command	Bash	xv6	Reason
echo hello > f ; cat < f	hello	hello	descriptor redirection
ls | grep READ	README	README line	pipe
(echo a; echo b) | wc	2 lines 2 words 4 bytes	2 2 4	group and list
echo one ; echo two	one then two	one then two	sequence
echo bg &	bg and prompt	bg and prompt	background
ls *	expands existing names	cannot open *	Bash pathname expansion
echo $HOME	/root	$HOME	Bash variable expansion
echo "a b"	a b	"a b"	Bash quote removal
mkdir d ; cd d ; ls ..	lists parent after changing directory	exec cd failed; lists from original directory	xv6 handles cd only as a whole input line
Up Tab Ctrl+R	history and completion	not implemented as Bash readline features	source and PTY inspection

There is an important distinction in the last directory case. A cd embedded in a semicolon list is passed through the xv6 execution parser and fails as an external command. Issuing cd d as its own input line does change the parent shell. From there a bare ls fails because xv6 has no PATH search, while /ls .. succeeds. For interactive work, history and completion are valuable additions. Basic quote handling would be a localized tokenizer improvement; full pathname expansion would also require directory traversal.

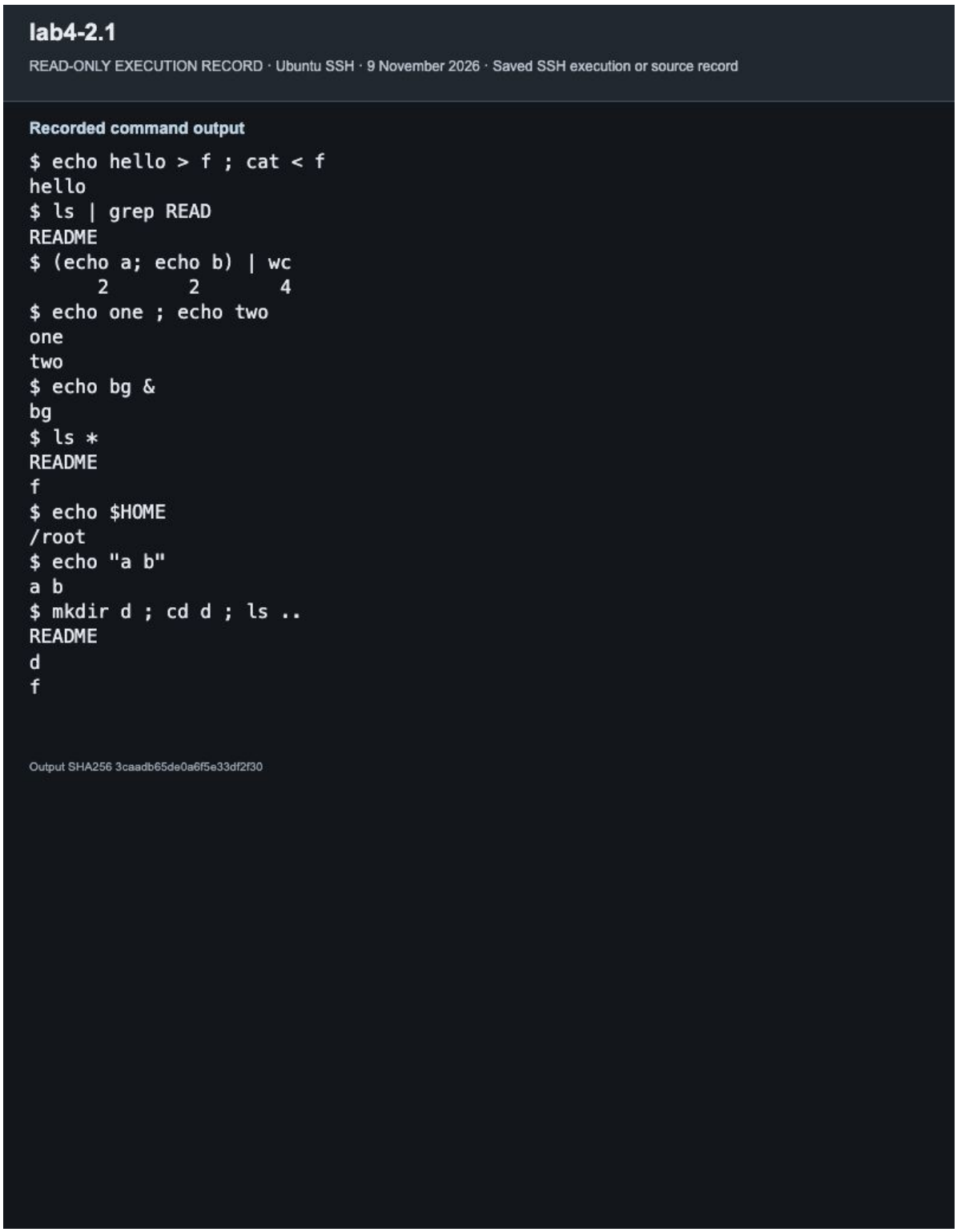


Figure 2.1 – The original comparison commands in Bash

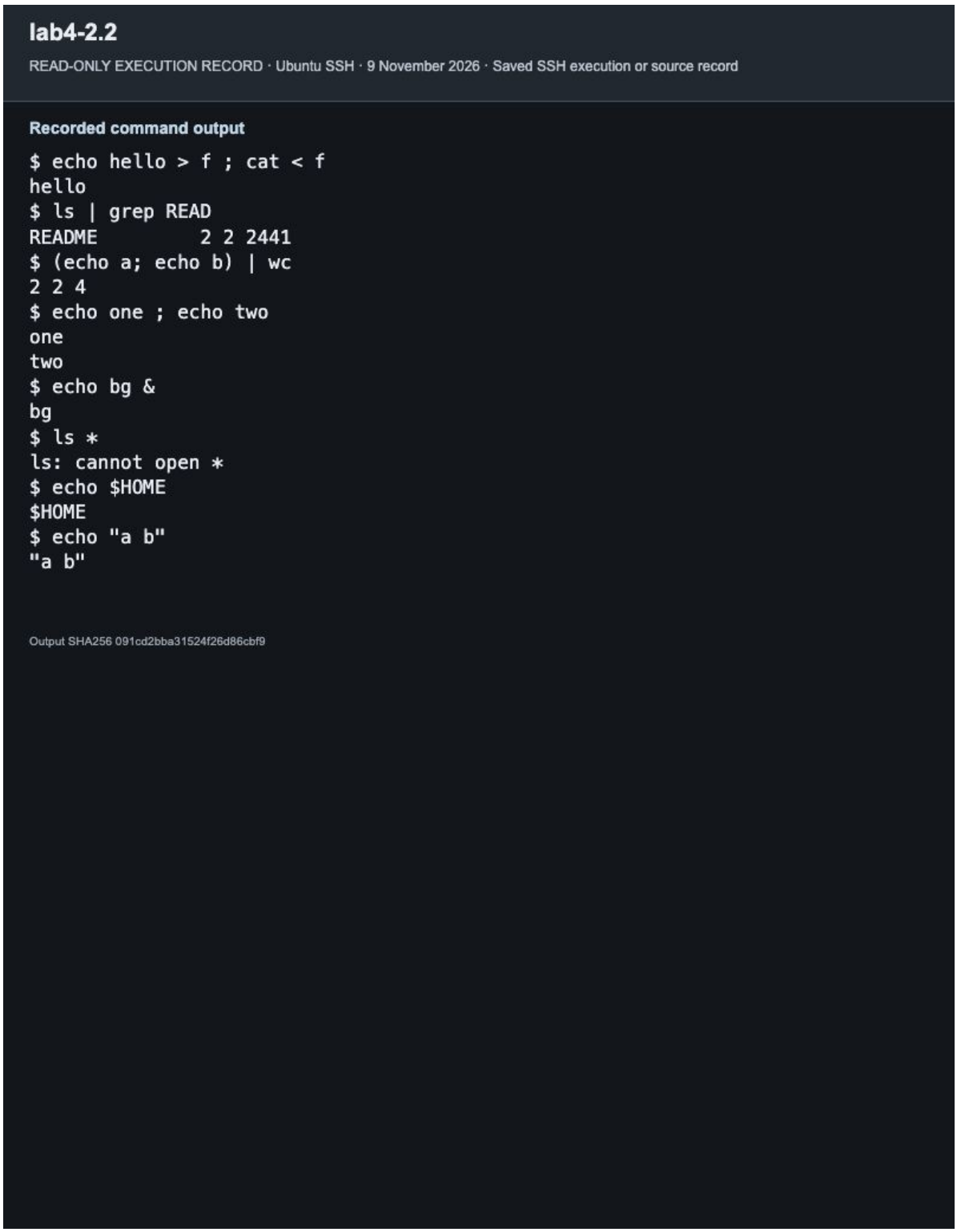


Figure 2.2 – The same comparison commands in xv6

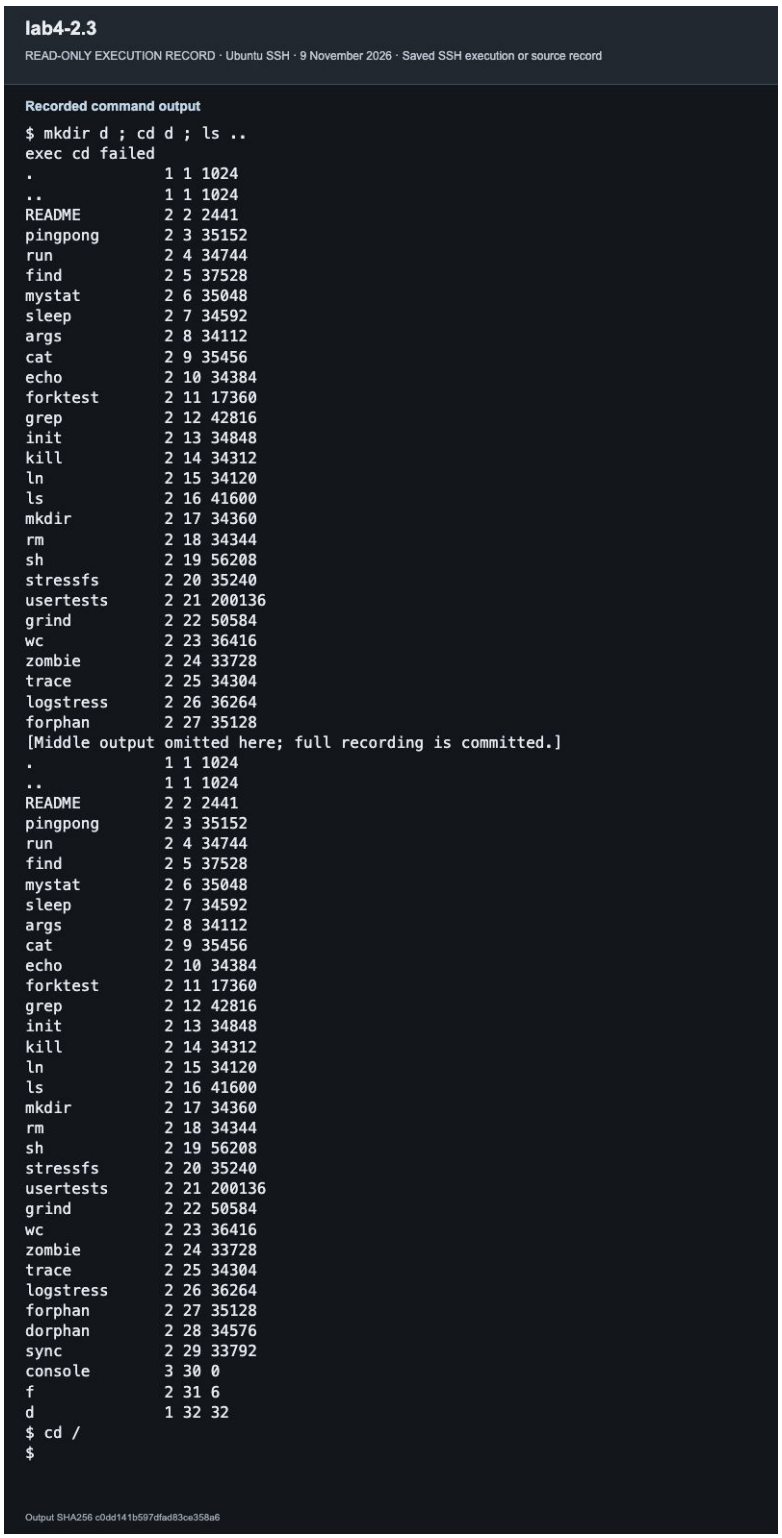


Figure 2.3 – Directory handling and absolute ls

Part 3 Who expands the argument vector

Both inspectors print only argc and argv. The Linux wildcard experiment in this additional test directory had four C files and one quoted x y operand, giving argc 6 including argv zero. Xv6 passed four arguments: args, the literal *.c, and two values containing the double quote marks. The shell, not the inspector or kernel, is responsible for expansion and syntactic quote removal. Repeated unquoted spaces delimit arguments in both shells; they do not become empty arguments.

lab4-3.1

READ-ONLY EXECUTION RECORD · Ubuntu SSH · 9 November 2026 · Saved SSH execution or source record

Recorded command output

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char **argv)
{
  printf("argc=%d\n", argc);
  for(int i = 0; i < argc; i++) printf("argv[%d]=[%s]\n", i, argv[i]);
  exit(0);
}

UPROGS
134:    $U/_pingpong\
135:    $U/_run\
136:    $U/_find\
137:    $U/_mystat\
138:    $U/_sleep\
139:    $U/_args\
```

Output SHA256 5b88a1a367b9e48733d2528b

Figure 3.1 – Args source and UPROGS entry



Figure 3.2 – Argument arrays from xv6



Figure 3.3 – Argument arrays from Linux

Part 4 Reading the shell

This version of user/sh.c has 499 lines. EXEC represents a plain command; REDIR is introduced by < or >; PIPE by |; LIST by a semicolon; and BACK by ampersand. Parentheses group a parsed command tree. Cd is recognized in the parent loop before forking and parsing a general command, so its directory change persists. The parser has no Bash parameter-expansion or globbing stage, explaining the literal * and $HOME results. The five command kinds are visible both in their definitions and in runcmd dispatch.

lab4-4.1

READ-ONLY EXECUTION RECORD · Ubuntu SSH · 9 November 2026 · Saved SSH execution or source record

Recorded command output

```
SHELL_KINDS
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Output SHA256 85336adfa25fecdca7bbcfd1

Figure 4.1 – Defined command kinds

lab4-4.2

READ-ONLY EXECUTION RECORD · Ubuntu SSH · 9 November 2026 · Saved SSH execution or source record

Recorded command output

```
SHELL_CASES
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
```

Output SHA256 c7ff3a10dba706a2e6ea4e8d

Figure 4.2 – Dispatch cases



Figure 4.3 – Parent shell implementation of cd

Conclusions

The two shells share process and descriptor mechanisms but differ before exec in parsing and expansion. The argument inspector isolates those differences. The additional cd-list test also shows why a small shell feature cannot be assumed to compose exactly as in Bash. Both requested programs and the integrated Makefile are submitted with the full xv6 source and raw console evidence.